Chapter 6

Making Software Respond Automatically

Sprint 6 – Enterprise Triggers That Stay Clean

Objective

The objective of this sprint was to understand the Enterprise Trigger design pattern in Salesforce and learn how to build clean, reusable, and maintainable Apex Triggers. The sprint focused on separating business logic from trigger logic by using Service Classes, allowing applications to scale efficiently while following Salesforce development best practices.

Understanding Enterprise Trigger Architecture

Enterprise Trigger Architecture is a design approach that improves the quality of Salesforce applications by keeping Apex Triggers simple and organized. Instead of writing all the business logic inside a trigger, the trigger only detects record events and passes control to a dedicated Service Class. This separation makes the application easier to maintain, reduces code duplication, and allows the same business logic to be reused across different processes.

Separation of Trigger Logic and Business Logic

A Trigger is responsible only for responding to database events such as record creation, update, or deletion. The actual business operations, validations, and calculations are handled inside Service Classes. This design improves readability, simplifies testing, and allows developers to modify business requirements without changing the trigger itself. Following this approach results in cleaner and more maintainable enterprise applications.

Clean Coding Practices in Salesforce

Building enterprise applications requires writing code that is easy to understand, reusable, and scalable. By following Enterprise Trigger Architecture, developers can reduce complexity, improve debugging, and ensure that multiple business processes can use the same Service Class. This approach follows Salesforce development best practices and supports long-term application maintenance.

Task 1 – Implementing the Enterprise Trigger

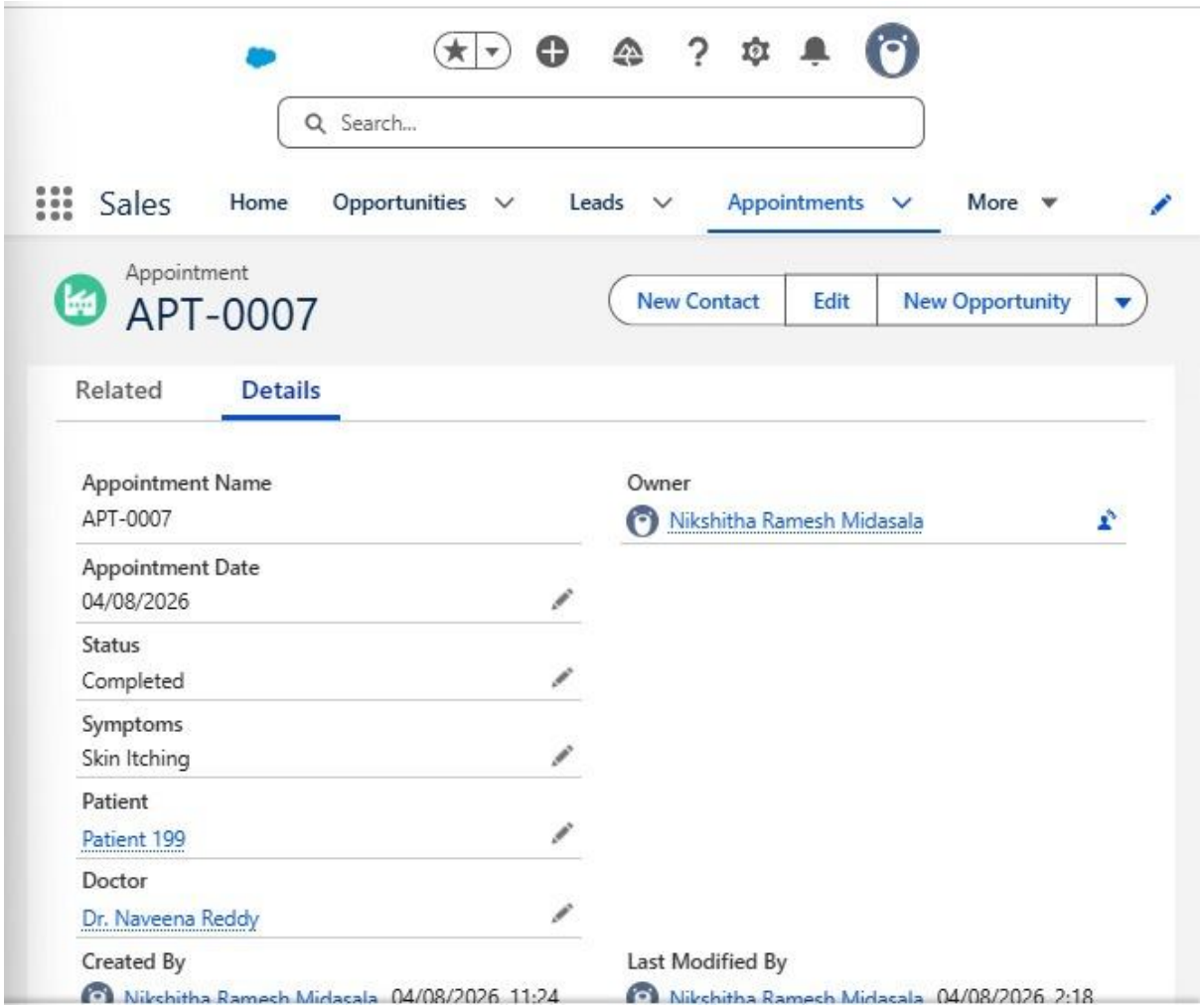
Objective

To implement an Enterprise Apex Trigger that automatically responds when an Appointment record is updated.

Work Performed

An Enterprise Trigger was created for the Appointment object to monitor record updates. Instead of implementing the business logic directly inside the trigger, the trigger delegated the processing to a Service Class. This ensured that the trigger remained lightweight while the Service Class handled the required business operations. The Appointment record was successfully updated, demonstrating the correct execution of the Enterprise Trigger architecture.

Appointment Record (APT-0007):



Task 2 – Automatic Prescription Creation

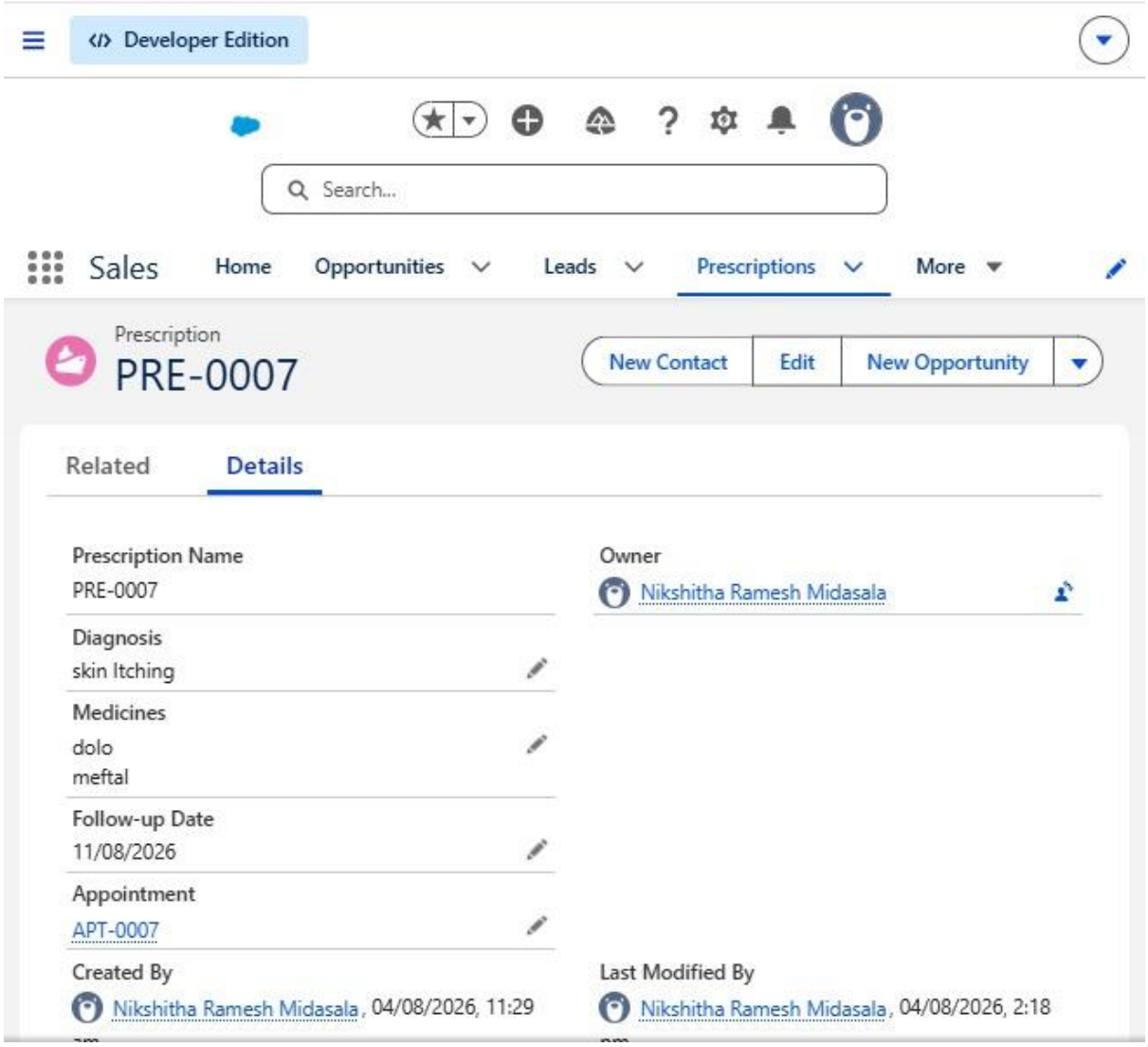
Objective

To automatically create a related Prescription record after an Appointment is marked as **Completed**.

Work Performed

After updating the Appointment status to **Completed**, the Enterprise Trigger invoked the Service Class. The Service Class automatically created a new Prescription record and populated the required information, including Diagnosis, Medicines, Follow-up Date, and the related Appointment reference. This demonstrated how business processes can be fully automated using Apex Triggers while keeping the implementation clean and reusable.

Prescription Record (PRE-0007):



Task 3 – Verifying Trigger Execution

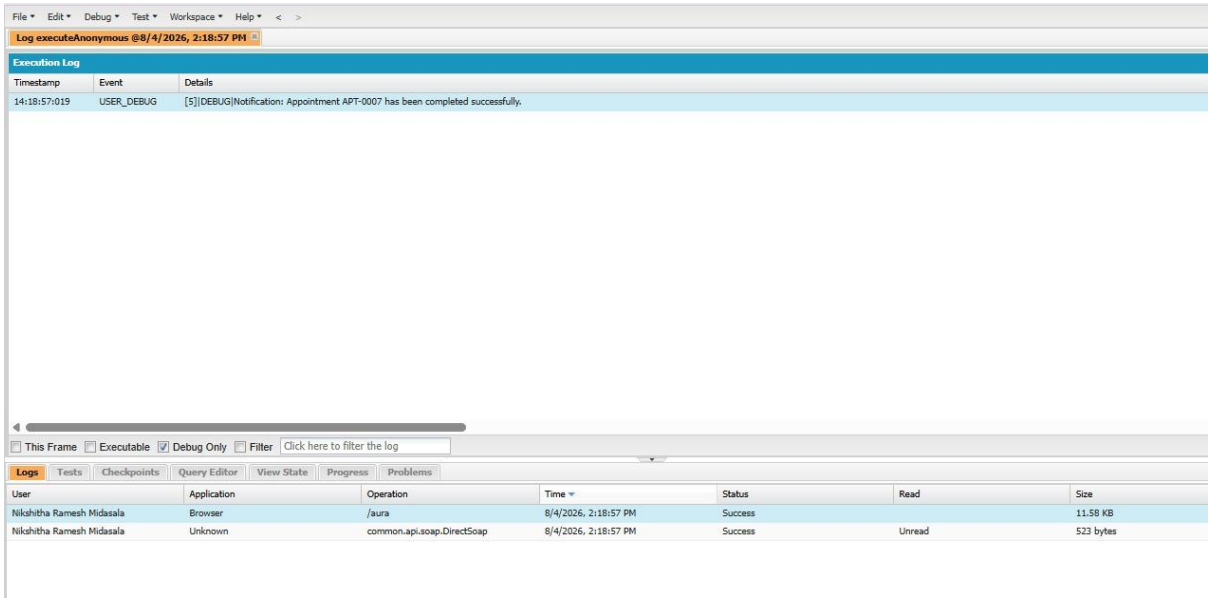
Objective

To verify that the Enterprise Trigger executed successfully after updating the Appointment.

Work Performed

The Developer Console was used to monitor the execution of the trigger. After the Appointment record was updated, the debug logs displayed a confirmation message indicating that the appointment had been completed successfully. This verified that the trigger executed correctly and the Service Class processed the required business logic without any errors.

Developer Console Debug Log:



Learning Outcomes

- Understood the importance of Enterprise Trigger Architecture in Salesforce development.
- Learned to separate Trigger logic from Business logic using Service Classes.
- Implemented automatic business process execution using Apex Triggers.
- Gained knowledge of writing clean, reusable, and maintainable Salesforce code.
- Verified trigger execution through Developer Console Debug Logs.
- Improved understanding of enterprise application development following Salesforce best practices.

Conclusion

This sprint provided a strong understanding of Enterprise Trigger Architecture and demonstrated how Apex Triggers can be designed using clean coding principles. By separating business logic into Service Classes, the application became more modular, reusable, and easier to maintain. The successful automation of Prescription creation after Appointment completion, along with verification through debug logs, reinforced the

importance of following Salesforce enterprise development standards for building scalable and maintainable applications.